Introduction to Angular 8 Workshop
Worldline Jan 2021

1 LAB PREPARATION 4

1.1 SOFTWARE INSTALLATION 4
1.2 REFERENCES 5
1.2.1 HTML 5
1.2.2 HTML DOM 5
1.2.3 CSS 6
1.2.4 Javascript 6
1.2.5 Angular 6

2 INTRODUCTION TO ANGULAR 6

2.1 WEB BASICS 6
2.2 INTRODUCING ANGULAR 6
2.3 ANGULAR ARCHITECTURE 6
2.4 CREATING A NEW PROJECT 6
2.5 ANGULAR PROJECT FOLDER STRUCTURE 7
2.6 WORKING WITH THE LAB CODE 7
2.7 WORKING WITH THE LAB EXERCISES 8

3 TEMPLATE SYNTAX AND BINDING 9

3.1 TYPESCRIPT: TYPES AND CLASSES 9
3.2 DECORATORS IN COMPONENTS AND MODULES 9
3.3 INTERPOLATION AND TEMPLATE EXPRESSIONS LAB 9
3.4 INTERPOLATION AND TEMPLATE EXPRESSIONS EXPLANATION 11
3.5 TYPING THE LAB CODE 11
3.6 PROPERTY BINDING LAB 11
3.7 PROPERTY BINDING EXPLANATION 12
3.8 EVENT BINDING LAB 12
3.9 EVENT BINDING EXPLANATION 13
3.10 TEMPLATE REFERENCE VARIABLES AND @VIEWCHILD LAB 13
3.11 TEMPLATE REFERENCE VARIABLES AND @VIEWCHILD EXPLANATION 14
3.12 CLASS BINDING LAB 14
3.13 CLASS BINDING EXPLANATION 15
3.14 EXERCISE: BINDING 15

4 PIPES 18

4.1 CREATING A NEW PROJECT 18
4.2 USING THE TITLECASE PIPE 18
4.3 INSTALLING THE BOOTSTRAP CSS FRAMEWORK 18
4.4 ADDING A DATE PICKER 19

4.5 Formatting date output with date pipe 19
4.6 Formatting currency output with the currency pipe 20
4.7 Formatting numeric output with the number pipe 20
4.8 Formatting object output with the JSON pipe 21
4.9 Creating a custom pipe 21
4.10 Using arguments with custom pipes 22

5 DIRECTIVES 23

5.1 Creating a new project 23
5.2 Generating a new component 23
5.3 Adding a child component 24
5.4 Creating and using a data model 24
5.5 Using NgClass directive 25
5.6 Using NgStyle directive 26
5.7 Adding / removing a single class with class binding 26
5.8 Using NgIf directive 26
5.9 Using NgFor directive 27
5.10 Using NgSwitch directive 28
5.11 Combining NgIf and NgFor directives 28
5.12 Adding Chrome Augury Dev Tool extension 28
5.13 Directives: Exercise 29

6 SIMPLE DEMO: COMPONENT COMMUNICATION 30

6.1 Creating a new project 30
6.2 Passing values from parent to child component via input decorator and property binding 31
6.3 Passing values from child back to parent via EventEmitter and event binding 31

7 STYLE ENCAPSULATION AND COMPONENT COMMUNICATION (IN MORE DETAIL) 32

7.1 Inline template and styles 32
7.2 Style encapsulation 33
7.3 Passing object to child component via input decorator 33
7.4 Returning object via EventEmitter to parent component 34
7.5 Using ChangeDetectionStrategy in child component 34
7.6 Component life cycle hooks 35
7.7 Using ngContent for content projection 36
7.8 Component: Exercise 36

8 TEMPLATE-DRIVEN FORMS 38

8.1 Combining event and property binding 38
8.2 Using NgModel and NgModelChange directives 39
8.3 Using NgModel alone for two-way binding 39
8.4 Form submission with NgSubmit 40
8.5 Using NgFor and NgValue for multiple drop box items 40
8.6 Working with validation classes: valid/invalid, pristine/dirty, touched/untouched 40
8.7 Using StockForm, template reference variables and detailed validation logic 41
8.8 Template-driven forms: Exercise 41

9 REACTIVE FORMS 44

9.1 Using FormControl directive 44

9.2 Using formGroup and formControlName with validators 45
9.3 Using FormBuilders 45
9.4 Adding detailed validation logic 45
9.5 Using FormGroup methods: setValue, patchValue and reset 45
9.6 Grouping form controls with FormArray 46
9.7 Reactive forms: Exercise 46

10 SERVICES 49

10.1 Modifying app to use a service 49
10.2 Creating a service and refactoring code to use service 50
10.3 Further refactoring to use service as the intermediary between two components 50
10.4 Creating and manually registering a service as a provider 51
10.5 Modifying main and child component to use same service 51
10.6 Configure service provider at child component level 51
10.7 Services: Exercise 52

11 OBSERVABLES 54

11.1 Callbacks and Promises: Basics 54
11.2 RxJS Basics 55
11.3 Refactor service and component to use observables 55
11.4 Refactoring to use observables naming convention 55
11.5 Using async pipe operator in template 56

12 HTTPCLIENT AND REST APIS 56

12.1 REST theory basics 56
12.2 Setup 57
12.3 Refactoring service to use HTTPClient lab 58
12.4 Refactoring data model for HTTP transport lab 58
12.5 Refactoring data model for HTTP transport explanation 58
12.6 Creating a proxy file to support CORS to access server on the same machine 58
12.7 Creating a proxy file for serving the app explanation 59
12.8 Adding HTTP headers lab 59
12.9 Adding HTTP headers explanation 59
12.10 Using observe and responseType properties lab 59
12.11 Using observe and responseType properties explanation 59
12.12 HTTP: Exercise 59

13 HTTP INTERCEPTOR 63

13.1 Generate and register authorization service lab 63
13.2 Generate and register authorization service explanation 63
13.3 Refactor code and service to use the Auth service lab 63
13.4 Refactor code and service to use the Auth service explanation 63
13.5 Create and set up interceptor in root module lab 63
13.6 Create and set up interceptor in root module explanation 64
13.7 Implement interception and response handling in interceptor lab 64
13.8 Implement interception and response handling in interceptor explanation 64

14 ROUTING 64

14.1 Creating a routing module and linking it to root module lab 64

14.2 MODIFY MAIN TEMPLATE TO LOAD COMPONENTS FOR ROUTES LAB 64
14.3 PROVIDING DEFAULT ROUTES IN ROUTING MODULE LAB 65
14.4 USING ROUTE PARAMETERS 65
14.5 ROUTE NAVIGATION WITHIN COMPONENT 65
14.6 ROUTE NAVIGATION IN TEMPLATE 65
14.7 PASSING QUERY PARAMETERS BETWEEN COMPONENTS 65
14.8 NAVIGATING TO THE SAME PAGE 65
14.9 GENERATING AUTHENTICATION GUARD AND UPDATING MODULES LAB 66
14.10 GENERATING DEACTIVATION GUARD AND UPDATING MODULES LAB 66
14.11 GENERATING ROUTE RESOLVER SERVICE AND UPDATING MODULES LAB 66
14.12 ROUTING: EXERCISE 67

15 DEPLOYING 69

1 Lab preparation

1.1 Software installation

1. Node/NPM:

https://nodejs.org/en/download/

2. TypeScript:

https://www.typescriptlang.org/#installation

To install, make sure that Node has been installed and type in a shell terminal:

```
npm install -g typescript
```

3. Install Angular CLI locally on your machine

https://angular.io/guide/setup-local

Type in a shell terminal (with admin privilege – Windows or sudo – Mac/Linux) :

```
npm install -g @angular/cli
```

You can check the current version of angular with:

```
ng --version
```

If you have serious problems at any point, you can uninstall with:

npm uninstall -g @angular/cli

https://stackoverflow.com/questions/39566257/how-to-uninstall-angular-cli

and then reinstall it again

4. Chrome

https://www.google.com/chrome/

We will use the Chrome browser to view, debug and work with our Angular apps. You can also use other modern browsers like Edge or Firefox, but the instructions in this lab are specifically geared towards Chrome.

5. Visual Studio Code (optional):

https://code.visualstudio.com/

This is the code editor that will be used during the workshop and instructions are provided with respect to this editor. However, feel free to use whatever code editor/IDE that you are familiar with. Other popular IDEs include Atom, Sublime Text, Webstorm, Brackets.

6. Notepad++ (optional)

https://notepad-plus-plus.org/downloads/

This is an enhanced version of the standard Notepad that comes with Windows – we will be using it to temporarily place code before copying and pasting into our IDE. Feel free to use whatever other text editor that you are familiar with, or you can stick with using the standard Notepad.

1.2 References

1.2.1 HTML

https://www.w3schools.com/tags/ref_byfunc.asp
https://www.w3schools.com/tags/ref_standardattributes.asp
https://www.w3schools.com/tags/ref_attributes.asp

1.2.2 HTML DOM

https://www.w3schools.com/js/js_htmldom.asp
https://www.w3schools.com/jsref/dom_obj_all.asp
https://www.w3schools.com/jsref/dom_obj_event.asp

1.2.3 CSS

https://www.w3schools.com/css/default.asp
https://www.w3schools.com/cssref/css_selectors.asp

1.2.4 Javascript

https://www.w3schools.com/js/default.asp
https://javascript.info/

1.2.5 Angular

https://angular.io/docs
https://angular.io/api

2 Introduction to Angular

2.1 Web Basics

Video link: https://youtu.be/Ii0Mte9O-Og

2.2 Introducing Angular

Video link: https://www.youtube.com/watch?v=piKF4g3tCXU

2.3 Angular Architecture

Video link: https://www.youtube.com/watch?v=yVtwBGg7Q7o

2.4 Creating a new project

Video link: https://youtu.be/1KQRMsdcujg

This application is built from scratch

From a shell terminal type:

```
ng new binding
```

Press enter to accept the defaults for all the questions that come up

The process of creating a project will take a while (depending on the speed of your broadband connection) as NPM will need to download and install all the Javascript modules that the Angular project needs to run properly. If the entire process takes longer than several minutes to complete, it is recommended that you generate the new projects for the remaining portion of the workshop during your break periods BEFORE the actual lab session itself so that you won't unnecessarily delay the progress of the workshop. We will be generating new projects in Section 4.

The newly created `binding` folder is the root project folder for the Angular App. Change to this directory in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

```
ng serve
```

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

2.5 Angular project folder structure

Video link: https://youtu.be/m4uaoW_KN4Y

Open up the root project folder using Visual Studio Code (or the IDE of your choice).

2.6 Working with the lab code

Video link: https://youtu.be/PK0BHDSZLe8

Create an empty folder (which I will call your personal folder) where you will generate and copy the project folders for the various Angular apps that you will build in this workshop. Give it any suitable name (e.g. `C:\myprojects`). **DO NOT** generate your project folders directly in your root drive C:\

You should be able to see a `workshop labs` folder in the zip file that you downloaded from GitHub. The various subfolders in `workshop-labs` contain files for the various Angular projects that we will be building, either from scratch or from an initial app configuration. The creation of the final app will proceed in a step-wise fashion so that the various features of the Angular framework can be clearly demonstrated at each step

The majority of subfolders in `workshop-labs` contains the following additional folders:

a) `xxx-start` (optional) – If the project is going to be built from an initial app configuration, the source code for this initial app will be here

b) `additions` – This contains the modifications to the various project source files in `xxx-start` as the app is gradually built up in a step-wise manner. For e.g. `xxx.v1.html, xxx.v2.html`, etc represents successive changes that are to be made to the `xxxx.html` in `xxx-start`.
c) `xxx-final` – The complete source code for the final working project for reference

For projects with an initial base app configuration, copy the `xxx-start` project folder to your personal folder that you created earlier.

Both `xxx-start` and `xxx-final` are missing the crucial `node_modules` subfolder which contain all the dependencies (Javascript libraries) that your Angular app needs in order to be built properly. These dependencies are specified in the `package.json` file in the root project folder. They are omitted in the project commit to GitHub because the number and size of the files are prohibitively large.

In order to build and run the app defined in either `xxx-start` or `xxx-final`, you will need these files. There are two ways to accomplish this:

- Copy the relevant project folder (`xxx-start` or `xxx-final`) to the working folder that you use for creating actual apps, and run `npm install` in a shell prompt in the root project folder. This can take quite a while to complete (depending on your broadband connection speed), and will need to be repeated for each individual project folder.

- An alternative is to reuse the dependencies of an existing complete project in the working folder as most of the projects for our labs use the same dependencies. In this case, copy over `xxx-start` and `xxx-final` to the working folder. Then copy the `node_modules` subfolder from the existing complete project over to here, and you should be able to immediately start the app. If you encounter any errors, this probably means that there are some missing dependencies or Angular version mismatch, in which case you will then have to resort to `npm install`.

The final working source code for the completed app can be found in `xxx-final`. If there was some issue in completing the step-wise construction of the app from `xxx-start`, you can copy this folder to your personal folder and install it with `npm-install` as described earlier.

2.7 Working with the lab exercises

Many of the labs here have exercises at the end of them to validate your understanding of the content. Depending on the cohort size, you will either work individually or as a group to solve the exercise. The proposed solution for this exercise will be broken down into a series of incremental steps, for which you will write very short code implementations for. If you are working as a group, everyone in the group will take turns to tackle each of these steps. You will all be allocated a period of time to study the exercise as a whole and also to produce the implementation for each incremental steps.

You will record your incremental code solutions in a Google Doc that will be shared with everyone: http://bit.ly/WorkshopSharedSpace

The assignment for solving each incremental step will be based on the participant name list accessible at: http://bit.ly/WorkshopParticipantList

I will use the StackBlitz online code editor to build up the solution app based on the incremental solutions provided by each of you.

https://stackblitz.com/

You will be able to access my app in real-time (and copy code out from it) although you will not able to modify it. Any code you write for your incremental solutions will be recorded in the shared Google Doc above, and I will simply transfer these to the StackBlitz app. You will work on the solutions in the IDE on your own machine.

Note: You can run multiple instances of the development server `ng serve` to serve up different Angular apps at the same time by running them on different ports. By default, just typing `ng serve` starts the server on the default port 4200. You could then run another instance, for e.g. in a different project or workspace folder, by typing:

ng serve --port=xxxx

where xxxx can be any free port on your machine, e.g. 9999

To see the list of used ports, type netstat -an into a command prompt shell.

3 Template syntax and binding

3.1 Typescript: Types and classes

Video link: https://youtu.be/km_wioge_K4

3.2 Decorators in components and modules

Video link: https://youtu.be/meRwSmVMSlM

3.3 Interpolation and template expressions lab

Video link: https://youtu.be/DYttBFEpMww

We will continue working with the `binding` project that we created earlier. The incremental changes to this project can be found in the `workshop-labs/binding/additions` folder.

Make the following changes:

`app.component.v1.html`

`app.component.v1.ts`

After you save these changes, notice that the Angular development server recompiles and updates the application automatically so that the new content is displayed in the browser view without the need to reload. This uses the hot module replacement feature of webpack, which is the main module bundler used in Angular.

Check `index.html` in `src` of the main project folder. Notice that it contains a <body> with a single <app-root> element. This is the main HTML for the app that is served to the browser from the Angular development server. If you right click on the Chrome browser view and select View Page Source from the context menu, you will be able to see the same HTML file; but with an additional 5 Javascript scripts referenced from it at the bottom. These 5 scripts together implement the functionality of the Angular app that will generate a live DOM from the static HTML to create the view that you see in your browser:

- runtime.js – Webpack runtime file
- polyfills.js – Polyfill scripts for supporting the variety of the latest modern browsers
- styles.js - Global style files bundled in one .js file
- vendor.js - Angular and other 3rd party vendor scripts bundled together
- main.js – The main application code

You can click on any of these script references to view their contents. Click on `main.js` and search for the string `Java and C++`. You should be able to see the compiled Javascript (ES5) for your AppComponent Typescript class. The largest of these Javascript files is `vendor.js`, which contains the compiled Angular library itself along with any other 3rd party library modules.

In the browser, select More Tools -> Developer Tools, and click on the Elements tab to switch to the Element views which provides a view of the DOM tree that the current view in the browser is rendered from. You will notice that the <app-root> element is now populated with content through the operation of these 5 scripts:

```
Elements  Console  Sources  Network  Performance  Memory  Ap
<!DOCTYPE html>
…<html lang="en"> == $0
  ▼<head>
      <meta charset="utf-8">
      <title>Binding</title>
      <base href="/">
      <meta name="viewport" content="width=device-width, initial-scale=1">
      <link rel="icon" type="image/x-icon" href="favicon.ico">
      <link rel="stylesheet" href="styles.css">
    ▶<style>…</style>
    </head>
  ▼<body>
    ▼<app-root _nghost-dvq-c11 ng-version="11.0.5">
        <h1 _ngcontent-dvq-c11> My first Angular app </h1>
        <h2 _ngcontent-dvq-c11> This app is hosted in Kuala Lumpur </h2>
        <p _ngcontent-dvq-c11> There are 3 languages that I like: Python, Java
        and C++</p>
        <p _ngcontent-dvq-c11>Programming is a fun activity</p>
      </app-root>
      <script src="runtime.js" defer></script>
      <script src="polyfills.js" defer></script>
      <script src="styles.js" defer></script>
      <script src="vendor.js" defer></script>
      <script src="main.js" defer></script>
    </body>
  </html>
```

Within the <body> element, you will see the <app-root> element, which is the top-level element of our Angular app in the DOM tree. Notice that there are the standard HTML elements such as <h1>, <h2> and <p> present; but these are augmented with attributes such as `_ngcontent-xxxx` and `_nghost-xxx`, which Angular uses as markers in order to perform operations such as binding and directives (to be covered later).

If you navigate to the Sources view, you can view all the 5 Javascript files mentioned earlier. Further at the bottom, if you navigate to `webpack://  .  src/app`, you will also be able to view the source code for your app (`app.component.html`, `app.component.ts`, etc) and you can also set Breakpoints here for debugging purposes (which we will cover later).

3.4 Interpolation and template expressions explanation

Video link: https://youtu.be/vwnpR9E45P0

3.5 Typing the lab code

Video link: https://youtu.be/4tj_OOxuWso

3.6 Property binding lab

Video link: https://youtu.be/uVV74rsHbvw

Make the following changes:

`app.component.v2.html`

`app.component.v2.ts`
`disabled` is one of the properties of a DOM Button Object
https://www.w3schools.com/jsref/dom_obj_pushbutton.asp

Experiment with changing the value of `disableFirstButton` in `app.component.ts` and notice how the binding causes the button to be disabled.

3.7 Property binding explanation

Video link: https://youtu.be/4rg_Rkr3NgI

3.8 Event binding lab

Video link: https://youtu.be/eQSv3tOPdNM

Make the following changes:

`app.component.v3.html`
`app.component.v3.ts`

In Chrome Developer tools, switch to Console view to see the output statements from the `console.log` in `app.component.ts`

Any run-time error messages typically appear in the Console view, so we should keep this open at all times when developing. We can also put `console.log` statements in our code to output messages here to help in debugging and tracking app execution.

The properties / methods of the MouseEvent which we access in `onFirstButtonClick`
https://www.w3schools.com/jsref/obj_mouseevent.asp

`event.target` returns the element that the event occurred on, which we need to cast to the appropriate type depending on the property that we wish to access. The code will still work without explicit casting, but the Typescript compiler will complain.

https://developer.mozilla.org/en-US/docs/Web/API/HTMLButtonElement
We cast to HTMLButtonElement to access the name and type

https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement
We cast to HTMLElement to access the text on the button itself

Make the next set of changes:

```
app.component.v4.html
app.component.v4.ts
```

Notice how each change in the value of `myfirstTextBoxText` due to keystroke events is immediately fed back via the interpolation expression `{{myfirstTextBoxText}}` in the view.

A list of most widely used events that can be used for event binding
https://www.positronx.io/useful-list-of-angular-7-event-types-for-event-binding/

3.9 Event binding explanation

Video link: https://youtu.be/sQQAn0T3L2A

3.10 Template reference variables and `@ViewChild` lab

Video link: https://youtu.be/G5JUljS8vT0

Make these set of changes

```
app.component.v5.html
app.component.v5.ts
```

- Try typing some content into the second text box
- Clicking on the first button or second button or typing something into the first text box registers the text typed in the second text box in the last statement at the bottom
- Clicking on the second button toggles the enabled/disabled status of the first button

The `static: true` used in the declaration of the element reference variable in `app.component.ts` is used to indicate that query results are resolved before change detection runs.

Notice that when we type in the second text box, nothing appears in the interpolation involving the template reference variable for this box at the bottom {{`mySecondTextBox.value`}}. This is because the new value for `mySecondTextBox.value` will only be detected and refreshed on a change detection cycle. The change detection cycle is only triggered on the execution of a template statement. This would be the case if:

- The first button is clicked `((click)="onFirstButtonClick($event)")` is executed
- The second button is clicked `((click)="onSecondButtonClick())` is executed
- Some text is typed into the first box
  `((input)="myfirstTextBoxText = $event.target.value"` is executed

Performing any of these 3 actions above will result in the new value updated in {{`mySecondTextBox.value`}}

Check also in the DOM tree in the developer console that the disabled property on the First Button is changed through property binding when the event handler function is invoked.

Make the next set of changes:

`app.component.v6.html`

Now typing some content into the second text box results in this content appearing immediately in the last statement at the bottom.

The `(keyup)="0"` is a dummy event capture to ensure that the template reference is constantly updated.
The "0" is a meaningless statement as blank statements are not allowed; what we want to do is to simply have a template statement to be executed so that it will immediately trigger the change detection cycle to update {{`mySecondTextBox.value`}} immediately.

Make the next set of changes

`app.component.v7.html`
`app.component.v6.ts`

- Type text into all 3 text boxes, then click the third button.
- The contents of text boxes are concatenated and displayed at the top, replacing the original paragraph: Programming is a fun activity.
- The contents of the second text box now becomes the text in the first button.

Here we are accessing and changing values of elements via their template reference variables

We can see HTML elements being accessed in 3 different ways in `onThirdButtonClick`:
a) From a component property bound via event binding (`myfirstTextBoxText`)
b) Using a property matched to an element via @ViewChild (`theSecondTextBox`)
c) Using a template reference variable passed as an argument to the event handler (`textBox: HTMLInputElement`)

We also change the content of various elements using their appropriate properties (`innerHTML` and `textContent`).

3.11 Template reference variables and `@ViewChild` explanation

Video link: https://youtu.be/ebhmqtcmRNw

3.12 Class binding lab

Video link: https://youtu.be/y1ayCxIfuIY

Make this set of changes:

`app.component.v1.css`

`app.component.v8.html`

Here, we apply a basic CSS styling rule to increase the font size of the first few sentences at the top.

Make the next set of changes:

`app.component.v7.ts`

`app.component.v9.html`

Type any sentence containing `cat` in the first text box and notice that the color of the 3rd sentence changes
Type any sentence containing `dog` in the second text box and notice that the color of the 4th sentence changes

Check from the DOM tree in the Elements view the effect of typing these two words on the class values of both affected paragraphs. Notice the values specified in the class binding (either `large` or `small`) overrides any existing class attribute value (`main`).

Note that class binding is a special form of binding provided by Angular. It is NOT a type of property binding, as there is no such property called `class` on a HTML DOM element (there is `classList` and `className` for working with attribute classes).
https://www.w3schools.com/jsref/dom_obj_all.asp

3.13 Class binding explanation

Video link: https://youtu.be/tQq0021IIuU

3.14 Exercise: Binding

Video link: https://youtu.be/RMr87v15dNM

Run npm install in `exercise-binding-without-start` and `exercise-binding-with-start`

Implement the following functionality in two solutions:

`exercise-binding-without-start:` WITHOUT the use of template reference variables (using only interpolation, property and event binding)

`exercise-binding-with-start:` with template reference variables ONLY (without interpolation and property binding, event binding can still be used).

a) Clicking on the button changes the button message and the displayed image. There are a total of 4 images (red, green, blue, yellow) and the button message indicates the next image to be displayed. The view cycles between these 4 images

b) Typing characters into the text box changes the message displayed below:
   - If no characters are entered, the message is : Nothing entered yet
   - Less than 6 characters: Entering a string
   - Less than 11 characters: Medium string
   - 11 characters or longer: Long string

Hints:
- The 4 image files are located in `/src/assets`

Action steps for `exercise-binding-without-start`:

No.	Action
1.	Add in image element referring to any one of the images in `src/assets/images` (Hint: https://www.w3schools.com/tags/tag_img.asp)
2.	Add in button element with suitable text (Hint: https://www.w3schools.com/tags/tag_button.asp)
3.	Add in an input text form with suitable text (Hint: https://www.w3schools.com/html/html_forms.asp)
4.	Add in a paragraph element with suitable text (Hint: https://www.w3schools.com/html/html_paragraphs.asp)
5.	Add in a string property `evalMessage` in AppComponent with a suitable initial value
6.	Perform interpolation using this string property to the paragraph element
7.	Create a method `changeImage`() in AppComponent with a single console log statement
8.	Use event binding to bind this method to the button element (Hint: https://www.w3schools.com/jsref/dom_obj_event.asp)
9.	Create an array of strings `imageNames` in AppComponent containing the names of all the 4 images in `src/assets/images`
10	Create a numeric counter variable which will keep track of the current image being displayed. Increment this variable in the `changeImage()` method.
11	Use property binding on the image element so that the image displayed will change according to the current value of the numeric counter variable. (Hint: https://www.w3schools.com/jsref/dom_obj_image.asp)
12	Modify `changeImage()` to ensure that the sequence of images displays repeats after the last image is displayed
13	Create an array of strings `actionNames` in AppComponent containing all 4 text messages that will change in the button every time it is clicked.
14	Use interpolation on the button element so that the button text displayed will change according to the current value of the numeric counter variable.
15	If the text messages are not appearing in correct synchronization with the sequence of images, make appropriate changes to AppComponent to ensure that this is the case.

16.	Create a method `evalInput()` in AppComponent with a single console log statement
17.	Register an event binding in the input text box to `evalInput`() and pass in the event object as a parameter
18.	Expand on `evalInput()` to access the length of the string typed into the text box
19.	Expand on `evalInput()` to put different text messages into `evalMessage` depending on the length of this string.

Action steps for `exercise-binding-with-start`:

No.	Action
1.	Replicate all the required HTML elements on the page from previous solution
2.	Add in `myImageTag`, `myButton`, `myTextBox` and `myParagraph` as template reference variables
3.	Recreate the same counter variable, array of strings `imageNames` containing the names of all the 4 images in `src/assets/images`, array of strings `actionNames` containing all 4 text messages that will change in the button every time it is clicked
4.	Create a method `changeImage()` in AppComponent and use event binding to bind this method to the button element. Increment the numeric counter variable that is used to keep track of the current image being displayed here. Make sure the value of this variable is reset when the last image is reached.
5.	Add ViewChild decorators with appropriate view query selectors to select `myImageTag`, `myButton` and `myParagraph` template reference variables. Give the corresponding ElementRef property names: `theImageTag`, `theParagraph`, `theButton`. Ensure that you import the necessary classes from the angular/core library.
6.	Use `theImageTag` property in `changeImage()` to change the image displayed, in conjunction with the numeric counter variable and array of the 4 image names. (Hint: https://www.w3schools.com/jsref/dom_obj_image.asp)
7.	Use `theButtonTag` property in `changeImage()` to change the text displayed on the button, in conjunction with the numeric counter variable and array of the 4 button text messages (Hint: https://www.w3schools.com/jsref/dom_obj_all.asp)
8.	Perform event binding on the input text box and use the template reference variable to pass in the contents of the textbox as a parameter to the `evalInput()` method, and then print the length of the contents.
9.	Expand on `evalInput()` to put different text messages into the contents of `theParagraph` property depending on the length of the contents. (Hint: https://www.w3schools.com/jsref/dom_obj_all.asp)

4 Pipes

4.1 Creating a new project

This application is built from scratch

From a shell terminal type:

ng new pipes

Press enter to accept the defaults for all the questions that come up

The newly created `pipes` folder is the root project folder for the Angular App. Change to this directory in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

4.2 Using the titlecase pipe

Make this set of changes:

`app.component.v1.html`

`app.component.v1.ts`

Verify that typing a lowercase string into the text field results in an identical string appearing below, but in title case

https://angular.io/api/common/TitleCasePipe

4.3 Installing the Bootstrap CSS framework

We will use Bootstrap as the CSS framework. In the project folder, type:

```
npm install bootstrap
```

To access the Bootstrap style-sheets, we add an import statement in the global `styles.css` file. We will first need to locate the exact location of the CSS file we need to import from the `node_modules` folder (this should be in `bootstrap/dist/css/).`

Make the following changes:

`styles.v1.css`

4.4 Adding a date picker

Make this set of changes:

`app.component.v2.html`

`app.component.v2.ts`

Check that the input fields now appear with Bootstrap styling, and the payment date appears appropriately in YYYY-MM-DD format.

4.5 Formatting date output with date pipe

Make this set of changes:

`app.component.v3.html`

Notice that the date now renders in this format to the right of the input field: `January 22, 2021`
This corresponds to the `mediumDate` format.

We use the DatePipe here to format the display of the date:
https://angular.io/api/common/DatePipe

This includes some additional options to further customize the format in which the date gets printed out.

Remember that you can use the name `date` for your property on the left hand side of the | symbol, while the name `date` on the right hand side refers to the pipe name itself. So, the pipe name and property name can be the same; they are distinguished by their order of appearance. To avoid confusion, you can alternatively use a different property name instead.

Angular will use the locale settings within the browser to figure out how it should format the date again, which may result in different date formats for the same pipe.

You can include other characters such as / + - in the string for the custom format options, e.g. :

```
mydate | date : 'd d  --- d / w w '
```

Example of using the following characters from the custom format option:

```
mydate | date : 'MMMM d,y'
```

```
December 12, 2020
```

4.6 Formatting currency output with the currency pipe

Make this set of changes:

```
app.component.v3.ts
```

```
app.component.v4.html
```

Notice that the payment amount now renders in this format: $234.00

https://angular.io/api/common/CurrencyPipe

To change the style of currency that gets printed out, we can use the currency code. So for example, to get Japanese yen using the currency codes

```
{{ amount | currency : 'JPY' }}
```

Currency codes automatically take care of the requirement for decimal points or otherwise. For example, with Japanese yen there is no decimal points as one yen is the smallest form of currency you can have. Thus there should not be any decimal point here even you enter a decimal into the input, and Angular automatically trims any that we enter.

4.7 Formatting numeric output with the number pipe

Make this set of changes:

```
app.component.v4.ts
```

```
app.component.v5.html
```

Notice that the height display renders exactly to 3 decimal places, automatically rounding up the actual decimal value entered in the text field.

https://angular.io/api/common/DecimalPipe

A very common use case is to trim off digits off the fractional part of the decimal number. The decimal pipe is useful for this purpose.

For example, this completely removes all the fractional digits:

```
{{ height | number : '1.0-0' }}
```

When doing trimming, the pipe function also does appropriate rounding up or rounding down.

4.8 Formatting object output with the JSON pipe

Make this set of changes:

`app.component.v5.ts`

`app.component.v6.html`

Notice that the `car` object displays in a readable format at the bottom.

The JSON pipe simply takes some kind of data structure (typically an object) and displays it in JSON format. It is primarily used for debugging, and rarely used in a production application.

https://angular.io/api/common/JsonPipe

For e.g. attempting to display an object property directly in interpolation syntax without the JSON pipe results in nothing meaningful being displayed. Try deleting the JSON pipe to see this.

4.9 Creating a custom pipe

In a terminal at the root project folder, type:

ng generate pipe convert

Make the following set of changes:

`convert.pipe.v1.ts`

`app.component.v6.ts`

`app.component.v7.html`

Here, the custom pipe `convert` has a trivial implementation of outputting the value it is applied to the console.

The pipe has one important method inside of it called transform; every time we make use of a pipe, this method is invoked. Keep in mind that the first argument into the transform method is likely to be an undefined value, as the argument is likely to be input from a field where the user has not typed in anything yet when the application initially bootstraps. So, a check needs to be made for this case through a simple conditional expression.

4.10 Using arguments with custom pipes

Make the following set of changes:

```
convert.pipe.v2.ts

app.component.v8.html
```

Notice now that the kilometers entry shows the correct conversion from the value entered in the Miles field. You can experiment with other arguments such as:

```
{{ miles : convert : 'm' }}
{{ miles : convert : 'cm' }}
```

We can also add in custom arguments to our custom pipe to further customize it. The syntax for doing this in the template is similar to the syntax for arguments for built-in pipes. Multiple custom arguments can be added, each separated by a colon

```
{{ miles : convert : 'm', 'another-arg, 'yet-another-arg'}}
```

The default signature of the transform method uses a rest parameter, args, for its 2nd argument in order to be able to accommodate multiple custom arguments:

https://www.javascripttutorial.net/es6/JavaScript-rest-parameters/
https://JavaScript.info/rest-parameters-spread

If you only expect to reach one additional custom argument, usually the best practice is to modify the default signature of the transform method to reflect this rather than use the rest parameter, which is what we do here:

```
transform (value: any, targetUnits: string): any {
```

Implementation of the logic for the additional custom argument will usually involve some form of multi conditional comparison, which can be achieved using a switch statement. In that case, you should put in a default case as well to handle for a situation when the pipe is used with an unrecognized custom argument value.

One simple way to handle this is to just return the passed value without any modification. A better way would be to throw an error.

5 Directives

5.1 Creating a new project

This application is built from scratch

From a shell terminal type:

ng new directives

Press enter to accept the defaults for all the questions that come up

The newly created `binding` folder is the root project folder for the Angular App. Change to this directory in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

5.2 Generating a new component

In a terminal at the root project folder, type:

ng generate component stock/stock-item

Notice that a specific folder is created with the name of the component (stock-item) and 4 files are generated in it:

- `stock-item.component.ts` (The Typescript class defining the component)
- `stock-item.component.html` (The HTML template for this component)
- `stock-item.component.css` (The stylesheet to apply to the template)
- `stock-item.component.spec.ts` (The skeleton unit test specification for the component)

Also notice that the component class name is imported in `app.module.ts` and also placed in the declarations array of the `@NgModule` metadata. This is automatically performed by the Angular CLI; if

you create the component class manually then you have to remember to perform this step, otherwise Angular will not be able to locate your component when it bootstraps the app.

5.3 Adding a child component

Basic interpolation and adding of a child component

`app.component.v1.html`

`app.component.v1.ts`

Notice that the `stock-item` template (`stock-item.component.html)` is now inserted into `app.component.html` at the point where the component selector for stock-item (`<app-stock-item></app-stock-item>`) appears.

Basic implementation and styling of child component

`stock-item.component.v1.ts`

`stock-item.component.v1.html`

`stock-item.component.v1.css`

Check that clicking the button causes it to become disabled (event and property binding combined together on the button tag).
Experiment with changing the values of the `price` and `previousPrice` properties so that the `positiveChange` property changes value and notice how the class value on the changes between positive and negative, with a different styling result applied.

Notice that `stock-item.component.html` contains interpolation, event binding, property binding and class binding.

The `ngOnInit` is a life cycle hook method that is required as the component implements the `OnInit` interface. There are many life cycle hook methods that are available in Angular, but the most common is `ngOnInit` which is called when the component is initialized. Therefore, the properties for the component can be initialized in either the constructor or in `ngOnInit`.

5.4 Creating and using a data model

In a terminal at the root project folder, type:

ng generate class model/stock

`stock.v1.ts`

Notice that for the generation of a class, there is no template or css. A unit test is still auto-generated however.

Refactoring the component and template to use the data model

`stock-item.component.v2.ts`

`stock-item.component.v2.html`

`stock-item.component.v2.css`

Here, we are refactoring the component and template to use the data model. The reason that we create a separate class (`stock.ts`) to hold the properties that were initially in `stock-item.component.ts` is because we would like to maintain the separation of data and its presentation, which is aligns with the philosophy of MVC. The component should ideally only contain logic related to how the data gets displayed in the HTML template, while the data itself should be stored and manipulated in a separate class.

5.5 Using NgClass directive

Make the following changes:

`stock-item.component.v3.ts`

`stock-item.component.v3.html`

Experiment with instantiating Stock object instance with different values for `previousPrice` and `price` (where both of them are differ by more than or less than 5.0) and view the effect on the template. View the class values for the holding the stock price in the Elements view in Chrome. Notice that there are multiple class names possible.

The `stockClasses` object contains a list of properties. The keys are intended to be used as class names (`positive, negative, etc`) while their matching values will be truthy / falsy values.

Keep in mind how Javascript evaluates truthy and falsy expressions

https://howtodoinjava.com/typescript/truthy-and-falsy/
https://stackoverflow.com/questions/35642809/understanding-javascript-truthy-and-falsy

Applying `stockClasses` to the `NgClass` directive will result in the addition of the class names of the keys whose values are truthy to the element. Notice that the existing class name of `price` is

not overridden; the new class names are simply added on. This is unlike the case of `[class]` binding in the previous example.

5.6 Using NgStyle directive

Make this set of changes:

`stock-item.component.v4.ts`

`stock-item.component.v4.html`

Experiment with instantiating Stock object instance with different values for `previousPrice` and `price` (where both of them are differ by more than or less than 5.0) and view the effect on the template. View the class values for the holding the stock price in the Elements view in Chrome.

We create the `stockStyles` object consist of a list of properties, where the key is valid CSS property (for e.g. `color`, `font-size`, etc) and the value is a valid value for that property (for e.g. `blue`, `12px`)

The NgStyle directive is based on inline styles:
https://www.codecademy.com/articles/html-inline-styles

5.7 Adding / removing a single class with class binding

Make this set of changes:

`stock-item.component.v5.html`

Here, we are adding / removing only one particular class with class binding

This is an extension of the class binding syntax that we saw in the previous example, where we can either add or remove one particular class but the existing class on the element (`price`) remains unchanged. With `[class.xxx]`, `xxxx` is added to the class name of the element if the expression that it is bound to is truthy.

5.8 Using NgIf directive

Make this set of changes:

`stock-item.component.v6.html`

Clicking on the Add to Favourite button should remove it

The *ngIf directive results in the element that it is attached to appearing in the DOM if the expression bound to the directive is truthy, and it is removed from the DOM otherwise. Note that this is very different from the HTML attribute hidden which removes a particular element from the browser view, but the element still remains in the DOM (https://www.w3schools.com/tags/att_hidden.asp)
The *ngIf directive on the other hand will remove the entire element from the DOM.

In the Elements view, you should see the particular button element being removed from the DOM and replaced with a comment.

Using NgIf else with ng-template

`stock-item.component.v7.html`

Experiment with instantiating `Stock` object instance with different values for `previousPrice` and `price` to view the effect on the template.

The *ngIf-else syntax is suitable for a situation where we wish for a particular element (or tree) to be attached to the DOM if the expression bound to the *ngIf is truthy, and for a different element (or tree) to be attached to the DOM otherwise. Notice the use of `ng-template` and a template reference variable to mark the alternative for the else statement.

5.9 Using NgFor directive

Make the following changes:

`stock-item.component.v5.ts`

`stock-item.component.v8.html`

You should now be able to see a listing of different stocks

Clicking on the Add to Favorite button of each particular stock container causes it to become disabled. You can see that particular button's disabled property become active in the Elements view. An output in the Console view that shows the index for that particular stock container in the overall array is also displayed.

*ngFor uses microsyntax. The variable after the keyword `let` is called a template input variable. The directive iterates over the array (`stocks`) and assigns the current item from the array to this variable (`stock`) for each iteration. The variable's scope is limited to a single instance of the repeated template. You can use the same variable name again in the definition of other structural directives.

A template input variable is not the same as a template reference variable (which always has a # prefix). Both have their own namespaces so you can have a template input variable with the same name as a template reference variable.

The `index` keyword of the NgFor directive context returns the zero-based index of the item in each iteration. You can capture the index in a template input variable (in this case, we use i) and use it in the template.

5.10 Using NgSwitch directive

Make the following changes:

`stock.v2.ts`

`stock-item.component.v9.html`

Notice the different messages (Buy now, Sell now, hold on) are displayed for each stock, depending on the price category they fall in.

5.11 Combining `NgIf` and `NgFor` directives

Make the following changes:

`stock-item.component.v10.html`

`stock-item.component.v6.ts`

Click on the Flip Share Market stocks buttons to switch between a listing of KLSE or NYSE stock containers.

Notice that we use a to wrap the ngIf within the parent ngFor directive. It is not possible to combine the ngIf and ngFor directives on a single element or component as Angular cannot determine which directive to apply first. You will need to nest the directives using a suitable grouping block.

5.12 Adding Chrome Augury Dev Tool extension

Augury is a widely used Developer Tool extension for debugging and profiling Angular applications inside the Google Chrome and Mozilla Firefox browsers.

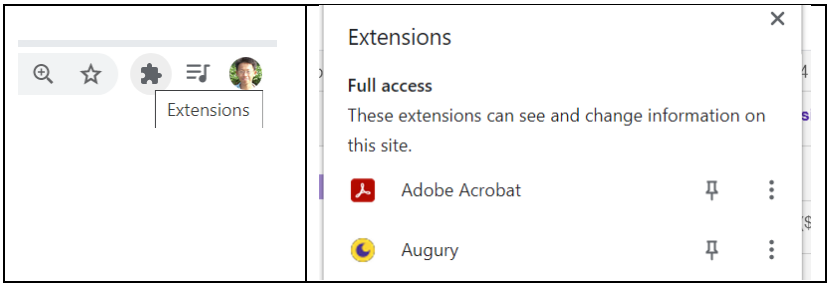
Go to:

https://augury.rangle.io/

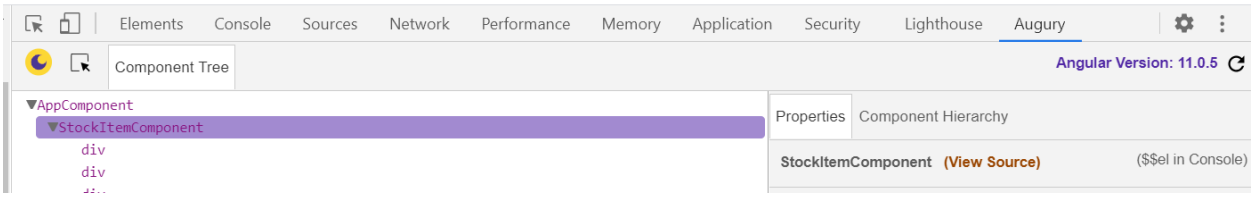
and click on Install on Chrome.

Then in the chrome web store page, click on Add to Chrome and confirm adding the extension.

In the Chrome browser, click on the extensions icon in the upper right hand corner and confirm that the Augury extension has full access to your browser.



Go back to the tab where the Angular development server is being accessed at (localhost:4200), close Developer tools view and reopen it. You should be able to see Augury as one of the main menu items. Click on it to navigate to the main Augury view.



This allows you to view the nesting of the components in the component tree as well, the state of the component (i.e. contents of its properties) and also access the source code of the individual components in webpack:// in the Sources tab of the Developer tools.

Select items from the Component Tree view and drill into them in more detail in the Properties / Component Hierarchy pane on the right. Remember to click on the refresh icon next to the Angular version at the upper right hand to refresh the component tree if necessary (i.e. if changes are not showing up, for e.g. when you click on the Flip Share Market stocks button)

5.13 Directives: Exercise

Video link: https://youtu.be/AvA-gptKLlk

Run npm install in `exercise-directives-start`.

Implement the following functionality:

a) Clicking on the `save string` button adds the current entry in the input text box into an internal list in the component.

b) Clicking on the `Show list of strings entered` button toggles between showing and hiding the strings in the list

c) The list of strings are styled in the following manner:
   - Strings with a length of 5 or longer appear in red. The class for this style is already in `app.component.css`. Use `NgClass` to accomplish this.
   - From the 5th string onwards in the list, the background color is light blue. Earlier strings do not have any background color. Use `NgStyle` to accomplish this.

Action steps for `exercise-directives-start`:

No.	Action
1.	Create a property `boxValue` and register event binding on the form to store the contents of the input text box directly into in the template expression itself (i.e. no need to call a component method).
2.	Create the following properties: an array of strings (`stringLog`), a string (`buttonMsg`), and a boolean (`showList`)
3.	Register an event binding on the second button to call a new method `onToggleListAppear`(), as well as an interpolation involving `buttonMsg`. In this new method, use `showList` and `buttonMsg` to toggle the message that appears on the button.
4.	Register an event binding on the first button (Save String) to call a new method `onSaveString`(), and in this method, store the contents of `boxValue` into `stringLog`.
5.	Use NgFor directive to display the list of items in `stringLog` at the end of the template. The position of the string in the list (start numbering from 1) and the string itself should be show in the list.
6.	Use NgStyle to set the background color for the list item to blue for the 5th string onwards in the list
7.	Use NgClass to set the class for the list item to the appropriate value so that the text turns red when the length of the item is 5 characters is more
8.	Use NgIf to control the visibility of the list depending on the value of `showList`

6 Simple Demo: Component communication

6.1 Creating a new project

This application is built from scratch

From a shell terminal type:

ng new parentchild

Press enter to accept the defaults for all the questions that come up

The newly created `parentchild` folder is the root project folder for the Angular App. Change to this directory in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

6.2 Passing values from parent to child component via input decorator and property binding

Create a component with:

```
ng generate new child
```

Make the following set of changes:

```
app.component.v1.html

app.component.v1.ts

child.component.v1.html

child.component.v1.ts
```

6.3 Passing values from child back to parent via EventEmitter and event binding

Make the following set of changes:

```
app.component.v2.html
```

`app.component.v2.ts`

`child.component.v2.html`

`child.component.v2.ts`

7 Style encapsulation and component communication (in more detail)

This project will start from the code base in `component-stuff/component-stuff-start`

Copy `component-stuff-start` to the working folder that you are currently using for the workshop labs, then type `npm install` in the root project folder. This will download the necessary dependencies to populate the `node_modules` folder.

Change to this root project folder in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

7.1 Inline template and styles

Make the following changes

`stock-item.component.v1.ts`

Notice that the template is now inlined in the component. The functionality and display of the app remains unchanged.

`stock-item.component.v2.ts`

Notice that the CSS stylesheet for the template is now inlined in the component. The functionality and display of the app remains unchanged.

7.2 Style encapsulation

Make this set of changes:

`app.component.v1.css`

`stock-item.component.v3.ts`

Notice only that the new style specified in `app.component.css` is only applied to the `<h1>` element in `app.component.html` (which has the class `name`) but not to the `<div>` in the template of the child component (`stock-item.component.html`) which also has the same class. This demonstrates styles are encapsulated within a component and only applicable to the template of that component. They are not inherited by any child components nested within the template.

You can check the Elements view to see how Angular uses special attribute markers in the tags to ensure that even though styling rules are included at the top of the generated DOM, but they are scoped to only a specific component that they are associated with.

Make the second set of changes: Disabling style encapsulation

`app.component.v1.ts`

With the new encapsulation property of `ViewEncapsulation.None`, the styling now cascades to apply to all nested components within `AppComponent`.

In the Elements view, there is now no longer any special attribute markers for the styles or tags in the top level root component, which means the style rules apply to all remaining templates in the DOM tree.

7.3 Passing object to child component via input decorator

Make the following set of changes:

`stock-item.component.v4.ts`

`app.component.v2.ts`

`app.component.v1.html`

Notice that clicking the Add to Favourite button correctly toggles the `favourite` property in the `stock` object causing it to become disabled; showing that the object was passed down successfully from the parent component to the child component

In the Augury view in Chrome, highlight `AppComponent` and try changing any some of the properties in `stockObj` in the view and press Enter when you are done. You will notice these changes are propagated down immediately to the child component and appear in the view.

7.4 Returning object via EventEmitter to parent component

Make the following set of changes:

`stock-item.component.v5.ts`

`stock-item.component.v1.html`

`app.component.v3.ts`

`app.component.v2.html`

Notice that clicking the Add to Favourite button correctly toggles the `favourite` property in the `stock` object causing it to disappear; showing that the same object is now returned successfully from the child component to the parent component via the <EventEmitter>

7.5 Using ChangeDetectionStrategy in child component

Make the following changes:

`stock-item.component.v6.ts`

`stock-item.component.v2.html`

`app.component.v3.html`

`app.component.v4.ts`

https://www.mokkapps.de/blog/the-last-guide-for-angular-change-detection-you-will-ever-need/

Experiment with clicking on all buttons.

Notice that clicking on Change Stock causes an immediate change, because the newly created object is being pushed to the StockItemComponent child component (with the change detection strategy set to `OnPush`). On the other hand, clicking on Change Price causes the state of the current `Stock` object to change instead of creating a new object, and therefore Angular's change detection does not run for this in the child component.

Change detection only runs when the object concerned changes state in StockItemComponent itself (due to `changeStockPrice` method being called) or when another newly created object is pushed to StockItemComponent due to clicking on Change Stock. However, all previous state changes to the `price` property are preserved regardless of whether they are due to invocation on `changeStockPrice` in StockItemComponent or `changeStockPrice` in AppComponent.

7.6 Component life cycle hooks

Make this set of changes

`app.component.v5.ts`

https://angular.io/guide/lifecycle-hooks

Refreshing the page should allow you to see the various calls made to the lifecycle hooks and the sequence in which they are called.
Click on all the buttons. Notice that all buttons will trigger DoCheck, AfterContentChecked and AfterViewChecked hook methods. This indicates that events from the child component (or any other component at a later point in the view hierarchy) will trigger DoCheck and all its accompanying methods.

DoChanged is not called here because `app.component.ts` does not have any @Input properties

Make this set of changes: Component life cycle hooks on child component

`stock-item.component.v7.ts`

Run this a few times, clicking on the various buttons of the parent (`app.component`) and child component (`stock-item.component`) templates. Notice:

- All the `Init` methods are only called once (`OnInit`, `AfterContentInit`, `AfterViewInit`) is only called once for both components
- The `AfterView` methods of the bottom-most child in the component hierarchy is called first, followed by the second bottom-most child and so all the way up to the root component.
- Clicking on the `Add to Favourite` buttons and the `Change Price` buttons in the child component (`stock-item.component`) result in cascade of `DoCheck` and `AfterContentChecked` method calls starting from the root component all the way down to the bottom-most child in the hierarchy.
- Clicking on these 2 buttons however DOES NOT result in `OnChanges` called, because there is no change in the component property binding from parent (`app.component`) to child
- Clicking on the `Change Stock` button in the parent component results in call to `OnChanges` in the child component, as there is now a change in the component property binding from parent to child (a new Stock object is created).

- Clicking on the `Change Price` button in the parent component DOES NOT result in a call to `OnChanges` in the child component, as the change is in the property of the object (`stockObj.price`) and not the object itself (`stockObj`), and therefore there is no change to in the component property binding from parent to child

7.7 Using ngContent for content projection

Make the following changes:

`stock-item.component.v3.html`

`app.component.v6.ts`

`app.component.v4.html`

Click on With Button 1 in the child template (`stock-item.component.html`), and notice that the corresponding method that is triggered is from the parent component (`app.component.v6.ts`)

The projected content into the child template appears at the location of the <ng-content> tags. Any event binding in this projected content will reference methods (`testMethod`) in the parent component, rather than the child component.

7.8 Component: Exercise

Video link: https://youtu.be/A2te1qMSk5A

Run npm install in `exercise-component-start`.

Implement the following functionality:

a) The app uses a data model based on the class `Employee.ts`. The `languages` and `os` properties are only meaningful in the context of the value of the `role` property. If the `role` property has the value `Admin`, then only the `os` property is used, and if the `role` property has the value `Developer`, then only the `languages` property is used.

b) There are 3 components in this app: `AppComponent`, `EmployeeDetailComponent` and `EmployeeFormComponent`.

c) `AppComponent` contains a radio button selector which allows the selection of either a Developer or Admin role. There are 2 additional buttons which can toggle the display of the employee form (handled by `EmployeeFormComponent`) and the list of employee details (handled by `EmployeeDetailComponent`)

d) Depending on the role value selected in the radio button, the employee form will show the relevant fields for that role (as explained in (a)).

e) When the `Add Record` button is clicked, the relevant info for a single employee is added to the main employee list that is maintained in `AppComponent`.

f) When the employee list is shown, only the relevant info for that particular employee is displayed in the context of that employee's role (as explained in (a)).

Hints:

- Create and maintain the main employee list (as an array of Employee objects) in `AppComponent`
- Use the `ngIf` directive in `AppComponent` along with two `boolean` properties to toggle the display of the `EmployeeFormComponent` and `EmployeeDetailComponent`.
- Pass the selected value in the radio button in `AppComponent` to `EmployeeFormComponent` via an `Input` property and component binding.
- Use this value in `EmployeeFormComponent` along with `ngIf` or `ngSwitch` directive to control the relevant input form fields to display.
- When the `Add Record` button is selected, create a new `Employee` object and pass it back to `AppComponent` via an `Output` property and component binding. Add this object to the main employee list in `AppComponent`
- Each `EmployeeDetailComponent` should ideally display details for only one employee record. Use the `ngFor` directive to iterate over the main employee list and then pass each individual `Employee` object to `EmployeeDetailComponent` via an `Input` property and component binding
- Just as in `EmployeeFormComponent`, use the `ngIf` or `ngSwitch` directive to control the relevant fields to display.

Action steps for `exercise-component-start`:

No.	Action
1.	Toggle the value of `showEmployeeForm` in a template expression in an event binding registered on the Show Employee Form button
2.	Use NgIf directive to determine whether the Employee Form appears in accordance to value of `showEmployeeForm`
3.	Set the `employeeRole` property to Developer if the Developer radio button is clicked on, and to Admin if the Admin radio button is clicked on. You can do this directly in the template expression of the event binding.
4.	Create an `@Input()` property called `myRole` in EmployeeFormComponent and pass the value of `employeeRole` through a binding in app.component.html
5.	Use interpolation to display the value of `myRole` in the main header for the EmployeeForm template
6.	Use the NgSwitch directive to determine whether to show the `OS used:` input text box or the `Languages developed in:` input text box depending on the value of `myRole`.
7.	Register an event binding on the name input text box so that its contents are stored into the corresponding property of the `myEmployee` object in EmployeeFormComponent. This assignment can be done directly in the template expression itself

8.	Repeat this event binding for all the other input text boxes in the form to store their values into their associated property in the `myEmployee` object.
9.	Create an `@Output()` property named `createdEmployee` in order to emit an object of class Employee, and initialize this property appropriately
10	Register an event binding on the Add record button to call a new method `onAddRecord()`. In this method, set the role properly on `myEmployee` and emit it via `createdEmployee`
11	Create a binding in app.component.html to received this emitted Employee object, and call a new method `onCreateEmployee()`. In this method, make the employee form disappear and put the Employee object into the `employeeList` array.
12	Create an `@Input()` property called `myEmployee` in EmployeeDetailComponent. Then use an NgFor directive to go through the `employeeList` and pass each array item through a suitable binding to `myEmployee`.
13	Use interpolation to display all the properties of `myEmployee` in the template of EmployeeDetailComponent.
14	Use an NgSwitch in the template to ensure that you show the correct field (`OS used:` or `Languages developed:`) depending on the employee role.
15	Toggle the value of the property `showEmployeeList` when the Show Employee List button is clicked, and use this to determine whether or not to display the list of employees in app.component.html

8 Template-driven forms

This project will start from the code base in `forms-template\forms-template-start`

Copy `forms-template-start` to the working folder that you are currently using for the workshop labs, then type `npm install` in the root project folder. This will download the necessary dependencies to populate the `node_modules` folder.

Change to this root project folder in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

8.1 Combining event and property binding

In a terminal at the root project folder, type:

ng generate component stock/create-stock

Make the following set of changes:

`app.module.v1.ts`

`create-stock.component.v1.ts`

`create-stock.component.v1.html`

`app.component.v1.html`

Type into the text field and notice that whatever you type is immediately displayed in the text box as well as in the sentence below

This is a basic example of capturing form input without using template-drive or reactive forms. We are using both event binding and property binding together in the same <input> element. Event binding is used to store the contents of each key stroke into stock.name and property binding is used to display these contents in the text field.

8.2 Using NgModel and NgModelChange directives

Make this change:

`create-stock.component.v2.html`

Check that the form still retains its previously functionality. Note that using these directives mean that we no longer need to know what is the specific input event and specific property to bind to for that given element (we just write `"stock.name=$event"` instead of `"stock.name=$event.target.value"`)

8.3 Using NgModel alone for two-way binding

Make this change:

`create-stock.component.v3.html`

Check again that the form still retains its previously functionality. This time we use the banana-in-the-boat syntax with NgModel

8.4 Form submission with NgSubmit

Make this set of changes:

`stock.v1.ts`

`create-stock.component.v2.ts`

`app.component.v1.ts`

`create-stock.component.v4.html`

Enter data into all the form controls, check the checkbox and click Create. You should see the contents of the fields replicated in the sentence at the bottom dynamically as you type content into the respective form controls. In the Console view, there should be a output showing the contents of the newly created Stock object holding the data from the corresponding form fields.

Note that NgModel and NgModelChange are still used separately in the event there is some additional operation to be performed besides a pure two-way binding between a component property and form control element value – for the case of stock price, a change in the value in the text box will invoke the `stockPrice` method (`(ngModelChange)="setStockPrice($event)"`) to perform an additional operation (`this.stock.previousPrice = price;`).

Note the use of the JSON pipe operator: `Stock Name is {{stock | json}}` to format the output

8.5 Using NgFor and NgValue for multiple drop box items

Make these changes:

`create-stock.component.v5.html`

`create-stock.component.v3.ts`

Check that data entry functionality for the existing form controls function as previously.

8.6 Working with validation classes: valid/invalid, pristine/dirty, touched/untouched

First set of changes: Adding CSS for color scheme for different states

`create-stock.component.v1.css`

`create-stock.component.v6.html`

Before typing anything into the form fields (if you already have, you can just refresh the page), view the classes attached to the 3 <input> elements: `stockName`, `stockCode` and `stockPrice`.

Notice that all 3 have identical values: `ng-untouched ng-pristine ng-valid`

Click in the stock name field, and start deleting characters from it. Notice one of the classes now changes to `ng-dirty` (indicating the control has been modified). Once you have removed all characters, notice that one of the classes changes to `ng-invalid` (indicating that the HTML `required` validation attribute was violated). Finally, clicking out from the field into any other part of the screen changes one of the classes to `ng-touched` (indicating that you have clicked in and out from the form control).

Type something into the stock code field, and again note the changes and change in color to pink.

Finally, click into the stock price field, and click out again without typing anything. Note the changes and the change in color to pink.

Make this next set of changes:

`create-stock.component.v2.css`

This time, we only set the color pink for the `ng-invalid` status class because typically we only want to highlight a form control when it is invalid, not when it has being modified or touched.

8.7 Using StockForm, template reference variables and detailed validation logic

Make the following set of changes:

`create-stock.component.v4.ts`

`create-stock.component.v7.html`

Start typing into the various form fields and verify that the appropriate validation message is displayed if there is an invalid entry (i.e. an entry that does not align with the HTML5 validation attributes added on the form controls).

Notice that if you attempt to submit the form (via the Create button) where one or more of the form controls are still in an invalid state, an error is logged to the Console view

Notice that the validation message for stock name is already shown when the form is displayed, while the validation messages for stock code and stock price are only shown when the user types in an invalid entry. This is due to the different conditional logic (`div *ngIf`) used for these form controls.

8.8 Template-driven forms: Exercise

Video link: https://youtu.be/vnQi2Dj3cOQ

Run npm install in `exercise-forms-template-start`.

This is the code base for the solution of `exercise-component`.

Implement the following functionality:

1. Add an additional property to `Employee`, which specifies an email (string type)
2. Revise the `App` template (`app.component.html`) and `EmployeeForm` template (`employee-form.component.html`) to use the template-driven approach to creating forms. The requirements are as follows:

- All fields are compulsory
- The employee name can only contain alphabets and spaces
- The employee email must follow a standard email format (xxx@yyy.zzz)
- If the employee is an Admin, the OS used can have a maximum length of 10 characters
- If the employee is a Developer, the languages used can have a minimum length of 4 characters
- Appropriate messages and styling are used for all fields to indicate validity as well as the type of errors (e.g. for the case of employee name, an empty field should result in a message saying the field is mandatory and an entry with non-alphabets should result in a message saying that only alphabets are accepted.
- Clicking the Add Record button with at least one invalid entry should result in an alert box indicating this situation.
- If the form is valid, then all entries are stored into the Employee object and transmitted from the `EmployeeForm` component in the usual manner.

Hints:
For the requirement of alphabets and spaces, as well as standard email format, you can use the `email` and `pattern=' ??? '` validator specified at https://angular.io/api/forms/Validators

Action steps for `exercise-forms-template-start`:

No.	Action
1.	Add an additional property email of type string to Employee class and update any code that initializes objects from this class. Import the required modules in the root module.
2.	Add in the CSS style rules to mark all valid input form control fields with a background of green, and all invalid fields with a background of pink.
3.	Use NgModel to obtain the employee name and store it in the corresponding property of `myEmployee`. Ensure you add in the validation requirements that it is: a) required b) only consists of letters and spaces. Wrap the input box in a div as well. For specifying validation on patterns, check: https://angular.io/api/forms/Validators#pattern Type: `pattern = ' ????? '` into the affected tag

4.	Add in validation logic to ensure that: When the user has not yet clicked in the name text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Employee name is mandatory` appears if there is no value in the box • `Employee name can only contain alphabets` if there are non-alphabets involved -
5.	Use NgModel to obtain the employee age and store it in the corresponding property of `myEmployee`. Ensure you specify that it is required. Wrap the input box in a div as well.
6.	Add in validation logic to ensure that an error message `Employee age is mandatory` is displayed if no age is entered in the age input box.
7.	Use NgModel to obtain the employee email and store it in the corresponding property of `myEmployee`. Ensure you specify that it is required. Wrap the input box in a div as well.
8.	Add in validation logic to ensure that: When the user has not yet clicked in the email text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Employee name is mandatory` appears if there is no value in the box • `Employee email is incorrect format` if it does not adhere to a standard email format
9.	For the case where the employee role is an admin, use NgModel to obtain the OS used: and store it in the corresponding property of `myEmployee`. Ensure you add in the validation requirements that it is: a) required b) max length of 10 characters
10.	Add in validation logic to ensure that if the user clicks in the OS used box but does not type anything, the error message `OS must be specified` appears.
11.	For the case where the employee role is an developer, use NgModel to obtain the languages developed in, and store it in the corresponding property of `myEmployee`. Ensure you add in the validation requirements: a) It is required b) It has a minimum length of 4 characters
12.	Add in validation logic to ensure that: When the user has not yet clicked in the languages developed text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Languages must be specified` appears if there is no value in the box • `Languages must be at least 4 characters` if it is shorter than 4 characters

13.	Create a NgForm and a NgSubmit for all the forms controls on this form, and link it to a submit button. This submission action will call a new method `createEmployee` in its associated component and pass the form as a parameter.
14.	Implement `createEmployee` in EmployeeFormComponent to check whether the employee form is valid. If it is, emit the `myEmployee` object as an event, otherwise provide an alert to the user that that the form invalid.

9 Reactive forms

Start from the code base in `forms-reactive\forms-reactive-start`. This is the final code base from the `forms-template` lab (the previous lab) and can also be found in `forms-template\forms-template-final`

Copy `forms-reactive-start` to the working folder that you are currently using for the workshop labs, then type `npm install` in the root project folder. This will download the necessary dependencies to populate the `node_modules` folder.

Change to this root project folder in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

9.1 Using FormControl directive

Make the following changes:

`app.module.v1.ts`

`create-stock.component.v1.html`

`create-stock.component.v1.ts`

Try entering a single value in the text input control to verify that it is working properly.

9.2 Using formGroup and formControlName with validators

Make the following changes:

`create-stock.component.v2.html`

`create-stock.component.v2.ts`

Try entering values for the various form controls and check that classes that are applied to them (`ng-valid`, `ng-dirty`, `ng-touched`, etc) in accordance to the specified requirements.

The complete list of built-in validators is available at:
https://angular.io/api/forms/Validators

9.3 Using FormBuilders

Make the following changes:

`create-stock.component.v3.ts`

Try entering values and check that there is no change in the form and validation functionality

9.4 Adding detailed validation logic

Make the following changes:

`create-stock.component.v4.ts`

`create-stock.component.v3.html`

Try entering values and check that the error messages that appear correspond to the logical expressions expressed in the various *ngIf directives.

9.5 Using FormGroup methods: setValue, patchValue and reset

Make the following set of changes:

`create-stock.component.v4.html`

`create-stock.component.v5.ts`

`stock.v1.ts`

`app.component.v1.ts`

Experiment with the buttons in the template. Note that the `setValue` method call requires the object being passed as a parameter (`stockFormCopy`) to be of the same shape as the `FormGroup` variable (in this case, `stockForm`): meaning it must have exactly a name, code and price property. To see this clearly, try and comment out either one of the `delete` statements from `loadStockFromServer()` and see the error messages that appear.

The `patchValue` method call will simply try to match the properties in the object being passed as a parameter to the controls of the `FormGroup` variable, automatically dropping properties from the parameter which are not used.

9.6 Grouping form controls with FormArray

Make the following set of changes:

`stock.v2.ts`

`create-stock.component.v6.ts`

`create-stock.component.v1.css`

`create-stock.component.v5.html`

Experiment with adding data to multiple Person form controls and verify that these are displayed appropriately and can also be removed.

9.7 Reactive forms: Exercise

Video link: https://youtu.be/KgJFzjdh_5U

Run npm install in `exercise-forms-reactive-start`.

This is the code base for the solution of `exercise-forms-template`.

Implement the following functionality:

a) Refactor the template-driven form approach in `employee-form.component.html` to use a reactive approach instead. Leave `app.component.html` in a normal or template-driven approach.

b) Maintain the same validation requirements as previously. In addition, ensure that the employee `age` property is between 20 and 80. Add appropriate validation error messages if this requirement is violated

Hints:

- You must include BOTH `ReactiveFormsModule` and `FormsModule` in your root module (`app.module.ts`) to use both normal (`app.component.html`) and reactive forms (`employee-form.component.html`) together in a single app
- If you have more than one validator to be applied to a form control with the `FormGroup`, ensure that these validators are grouped into an array (e.g. [`Validators.required`, `Validators.min(20)`, …etc])
- For the requirement of alphabets and spaces, as well as standard email format, you can use the `email` and `pattern='??? ' validator` specified at https://angular.io/api/forms/Validators

Action steps for `exercise-forms-reactive-start`:

No.	Action
1.	Import the required modules in the root module.
2.	Create an `employeeForm` as a FormGroup using the FormBuilder in EmployeeFormComponent. You may need to import the required classes. The FormGroup is initially empty; you do not need to add anything into it yet.
3.	Add the employee `name` field to this FormGroup with the appropriate validators: a) required b) only consists of letters and spaces.
4.	Add the employee `age` field to this FormGroup with the appropriate validators: a) required b) between 20 and 80 Give it an initial value 0.
5.	Add the employee `email` field to this FormGroup with the appropriate validators: a) required b) it must a follow an email format Give it an initial value of an empty string.
6.	Add the employee `os` field to this FormGroup with the appropriate validators: a) required

	Give it an initial value of an 'Linux'.
7.	Add the `languages` developed field to this FormGroup with the appropriate validators: a) required b) min length of 4 Give it an initial value of 'Java'.
8.	Remove all the existing form controls in employee.form.component.html. Make the entire form use a FormGroup Directive.
9.	Add in again all the input form controls for `name`, `age` and `email`, but this time using `formControlName` style for reactive forms
10.	Add in again the input form controls for `os` used and `languages` developed in using NgSwitch based on employee role, again using `formControlName` style for reactive forms
11.	Add in validation logic to ensure that: When the user has not yet clicked in the name text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Employee name is mandatory` appears if there is no value in the box • `Employee name can only contain alphabets` if there are non-alphabets involved
12.	Add in validation logic to ensure that: When the user has not yet clicked in the age input box, no message appears. Once the user has clicked in the input box and started typing, the following error messages appear • `Age is required` is displayed if no age is entered in the age input box. • `Age must be 20 or higher` is displayed if the age is less than 20 • `Age must be 80 or lower` is displayed if the age is more than 80
13.	Add in validation logic to ensure that: When the user has not yet clicked in the email text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Email is required` appears if there is no value in the box • `Email is incorrect format` if it does not adhere to a standard email format
14.	Add in validation logic to ensure that if the user clicks in the OS used box but does not type anything, the error message `OS must be specified` appears.
15.	Add in validation logic to ensure that: When the user has not yet clicked in the languages developed text box, no message appears. Once the user has clicked in the textbox and started typing, the following error messages appear • `Languages is required` appears if there is no value in the box • `Languages must be 4 characters or more` if it is shorter than 4 characters
16.	Add in a submit button with the text "Add Record" which will call the method

	`createEmployee()` in EmployeeFormComponent when clicked. Provide a correct implementation of this method.

10 Services

Start from the code base in `services-basic\services-basic-start`. This is the final code base from the `forms-template` lab and can also be found in `forms-template\forms-template-final`

Copy `services-basic-start` to the working folder that you are currently using for the workshop labs, then type `npm install` in the root project folder. This will download the necessary dependencies to populate the `node_modules` folder.

Change to this root project folder in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

10.1 Modifying app to use a service

In a terminal at the root project folder, type:

ng generate component stock/stock-list

Make the following changes:

`stock-list.component.v1.ts`

`stock-list.component.v1.html`

`app.component.v1.ts`

`app.component.v1.html`

`stock-item.component.v1.html`

10.2 Creating a service and refactoring code to use service

In a terminal at the root project folder, type:

ng generate service services/stock

Notice that the service class has the word `service` appended to the end of it automatically by Angular during generation.

Make the following changes:

`stock.service.v1.ts`

`stock-list.component.v2.ts`

Verify that the output rendered remains the same

Basics of higher order functions
https://www.freecodecamp.org/news/a-quick-intro-to-higher-order-functions-in-javascript-1a014f89c6b/

Note the use of arrow functions in the stock service implementation
https://www.sitepoint.com/es6-arrow-functions-new-fat-concise-syntax-javascript/
https://zendev.com/2018/10/01/javascript-arrow-functions-how-why-when.html
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

It uses the `find` and `push` methods of an array
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/find
https://www.geeksforgeeks.org/javascript-array-find-method/

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/push
https://www.geeksforgeeks.org/javascript-array-push-method/

10.3 Further refactoring to use service as the intermediary between two components

Make the following changes:

`create-stock.component.v1.html`

`create-stock.component.v1.ts`

Notice that creating a new stock in the form now immediately results it in being added to the stock list at the top, and a success message displayed at the top of the form. Attempting to create a new stock that has an identical code with an existing stock in the stock list at the top results in an error message being displayed and the associated stock is not added. Also clicking on the button on any of the stock items flips the value between the two strings: Remove from Favorite and Add to Favorite

The domain data model and its associated logic is now centralized in the StockService.

10.4 Creating and manually registering a service as a provider

In a terminal at the root project folder, type:

ng generate service services/message

`message.service.v1.ts`

`app.module.v1.ts`

Notice that in this case, because the MessageService only has @Injectable() (without the `providedIn: 'root'` metadata, it must be registered as a provider in NgModule

10.5 Modifying main and child component to use same service

Make the following changes:

`app.component.v2.ts`

`app.component.v2.html`

`create-stock.component.v2.ts`

`create-stock.component.v2.html`

Experiment with creating new stocks in the Create Stock Form part of the page. Notice that the informational messages regarding the success or otherwise of the stock creation is shown in the template of the main component (`app.component.html`) as well as CreateStock (`create-stock.component.html`). The `CreateStock` component and the `App` component communicate using the Message service, for which there is only one single instance

10.6 Configure service provider at child component level

`create-stock.component.v3.ts`

Notice now that the informational messages regarding the success or otherwise of the stock creation is shown only in `create-stock.component.html`. This is because the instance of `MessageService` created here is DIFFERENT from the instance of the `MessageService` created in `App` component. Therefore the changes made to the `MessageService` instance registered with `CreateStock` DO NOT influence the `MessageService` instance of `App` component and vice versa.

10.7 Services: Exercise

Video link: https://youtu.be/yO0LAiRi8wA

Run npm install in `exercise-services-start`.

This is the code base for the solution of `exercise-forms-template`.

Implement the following functionality:

a) Create a new service `EmployeeService` and a new component `EmployeeDisplay`. Refactor the app code base so that this service maintains the list of employees while the `EmployeeDisplay` displays the details of the employees via the `EmployeeDetail` component. Both `EmployeeDisplay` and `EmployeeForm` component interact directly with this service (instead of the App component) to retrieve the list and also add new entries to the list.

b) Create another service `HighlightService`. In the `App` component, allow the user to specify several in-demand languages / OS and the employee displayed by `EmployeeDetail` will be highlighted in a specific manner if the employee has those specific languages / OS skills. You can use any particular styling you want for the highlight (e.g. change background color, emphasize text, increase font size, use borders for div, etc).

Hints:
You may find using the `split` and `indexOf` methods available for Javascript strings helpful in your solution:

https://www.w3schools.com/js/js_string_methods.asp
https://www.w3schools.com/jsref/jsref_split.asp

Action steps for `exercise-services-start`:

No.	Action
1.	Create two new services `EmployeeService` and `HighlightService` as well as a new component `EmployeeDisplay` using the Angular CLI
2.	Create an array of Employee objects `employeeList` in EmployeeService and initialize it to an empty array in the constructor

3.	Add the following methods to EmployeeService:
	`getEmployees()` – returns `employeeList`
	`addEmployee()` – adds a single Employee object to `employeeList`

4.	Create an array of Employee objects `employeeList` in EmployeeDisplayComponent and inject the EmployeeService service.
5.	When the component is initialized, get the array of Employees from EmployeeService and store it into `employeeList`.
6.	Inject the EmployeeService into EmployeeFormComponent. For the @Output property, change the `EventEmitted` type to `boolean` and refactor the rest of the code accordingly.
7.	Then refactor the `createEmployee` method to add the newly created form to EmployeeService.
8.	Refactor AppComponent so it no longer maintains an `employeeList` array of Employee objects (this role is now taken over by EmployeeService). It will still hide the EmployeeForm once data for a form has being submitted.
9.	Refactor app.component.html so that it uses EmployeeDisplayComponent to display the list of employees
10.	Refactor employee-display.component.html so it now displays `employeeList` from EmployeeDisplayComponent
11.	Add a public property of String type called `importantStuff` to HighlightService. Create a method called `checkImportant` which takes a single string parameter `skills`. The string `importantStuff` may contain smaller substrings separated by spaces. If any of these substrings are contained in `skills`, the method returns true otherwise it returns false. You can use the `split` and `indexOf` methods available for Javascript strings: https://www.w3schools.com/js/js_string_methods.asp https://www.w3schools.com/jsref/jsref_split.asp You may want to write a separate small Typescript program and compile and run it with node.js to test this method before including it in the service.
12.	Inject HighlightService into AppComponent. Refactor app.component.html so that the form in it now includes a new input text box to get a list of important OS or languages to highlight. The value of this textbox is then stored into the `importantStuff` property of HighlightService. This text box is template-driven (uses `NgModel`)
13.	Create a style rule in employee-detail.component.css that targets the class value `important` and changes its font-size, color and background-color to some appropriate values.

14.	Inject HighlightService into EmployeeDetailComponent. In the initialization of this component, set a variable `stringToCheck` to either the languages or the os depending on the role of the employee.
15.	In EmployeeDetailComponent, create a new object property `employeeClasses`. We will be using this property with the `NgClass` directive in the template of EmployeeDetailComponent to determine the style to apply to the employee details. In the initialization of this component, ensure that `employeeClasses` has a single property with the key `important` and a boolean value. Use the HighlightService appropriately to determine whether the string entered in the textbox in app.component.html for a list of important OS or languages to highlight is matched by either the language or os of the current employee (given by `stringToCheck`).
16.	Refactor employee-detail.component.html to use the `employeeClasses` object with the `NgClass` directive to style the employee appropriately.

11 Observables

Start from the code base in `observables-basic\observables-basic-start`

Copy `observables-basic-start` to the working folder that you are currently using for the workshop labs, then type `npm install` in the root project folder. This will download the necessary dependencies to populate the `node_modules` folder.

Change to this root project folder in an independent shell terminal (or open the folder using VS-Code and open a terminal here) and type:

ng serve

to start Angular's customized web server that will serve your Angular app dynamically

Open a browser tab at:

http://localhost:4200/

to see the default landing page of the deployed app.

11.1 Callbacks and Promises: Basics

Basics of higher order functions

https://www.freecodecamp.org/news/a-quick-intro-to-higher-order-functions-in-javascript-1a014f89c6b/

Callback functions
https://www.freecodecamp.org/news/javascript-callback-functions-what-are-callbacks-in-js-and-how-to-use-them/
https://codeburst.io/javascript-what-the-heck-is-a-callback-aba4da2deced

Callback hell
http://callbackhell.com/

Promises
https://www.freecodecamp.org/news/javascript-es6-promises-for-beginners-resolve-reject-and-chaining-explained/
https://www.digitalocean.com/community/tutorials/javascript-promises-for-dummies

Chaining promises
https://www.javascripttutorial.net/es6/promise-chaining/

11.2 RxJS Basics

https://www.learnrxjs.io/learn-rxjs/concepts/rxjs-primer

11.3 Refactor service and component to use observables

Make this changes:

```
stock.service.v1.ts

stock-list.component.v1.ts

create-stock.component.v1.ts
```

Check that the app is fully functional now after being refactored

https://www.learnrxjs.io/learn-rxjs/operators/creation/of
https://rxjs-dev.firebaseapp.com/api/index/function/throwError

11.4 Refactoring to use observables naming convention

Make this changes:

```
stock-list.component.v2.ts
```

Check again that the app is fully functional now after refactoring

11.5 Using async pipe operator in template

Make this change:

`stock-list.component.v1.html`

Check again that the app is fully functional now after refactoring

12 HTTPClient and REST APIs

Start from the code base in `http-basic\http-basic-start`. This is the final code base from the `observables-basic` lab and can also be found in `observables-basic\observables-basic-final`

12.1 REST theory basics

https://restfulapi.net
https://www.restapitutorial.com

https://www.tutorialspoint.com/json/index.htm

There are several sites that provide online REST APIs that return dummy data that will help you in developing / testing HTTP calls from your client-side Angular app:

https://jsonplaceholder.typicode.com/
https://jsonplaceholder.typicode.com/guide/

https://dummyapi.io/

You can use the Postman app to perform REST API calls to a backend service for testing and development work:

https://www.postman.com/downloads/

If you use the browser based version of POSTMAN, you will not be able to test the local backend service here due to the CORs issue. You will need to use the POSTMAN client that can be downloaded and installed to your machine.

12.2 Setup

In the folder `http-basic\basic-server`
run:

npm install
node index.js

to start the back-end server. This basic server exposes 3 APIs:

- GET on `/api/stock` to get a list of stocks
- POST on `/api/stock` with the new stock as a body to create a stock on the server
- PATCH on `/api/stock/:code` with the stock code in the URL and the new favorite status in the body of the request, to change the state of favorite for the particular stock.

Type this URL into the address bar of the browser to test out the GET API:

http://localhost:3000/api/stock

Confirm the return response of this JSON document:

[{"name":"Test Stock Company","code":"TSC","price":85,"previousPrice":80,"exchange":"NASDAQ","favorite":false}, {"name":"Second Stock Company","code":"SSC","price":10,"previousPrice":20,"exchange":"NSE","favorite":false}, {"name":"Last Stock Company","code":"LSC","price":876,"previousPrice":765,"exchange":"NYSE","favorite":false}]

The GET API also supports query parameters. Try accessing the backend GET API with a simple query parameter:

http://localhost:3000/api/stock?q=test

This returns a list of stocks whose name contains the string (in this case 'test') specified in the q parameter.

`[{"name":"Test Stock Company","code":"TSC","price":85,"previousPrice":80,"exchange":"NASDAQ", "favorite":false}]`

Try again with a different parameter value:

http://localhost:3000/api/stock?q=stock

Again, this returns a list of stocks whose name contains the string (in this case 'stock') specified in the q parameter.


```
[{"name":"Test Stock
Company","code":"TSC","price":85,"previousPrice":80,"exchange":"NASDAQ","favo
rite":false},{"name":"Second Stock
Company","code":"SSC","price":10,"previousPrice":20,"exchange":"NSE","favorit
e":false},{"name":"Last Stock
Company","code":"LSC","price":876,"previousPrice":765,"exchange":"NYSE","favo
rite":false}]
```


12.3 Refactoring service to use HTTPClient lab

Make the following set of changes:

`app.module.v1.ts`

`stock.service.v1.ts`

Notice that we type the return value from the `http.get` method call to `<Stock[]>`
We also provide only the URL path as the argument to the various http method calls, while the precending actual domain name and port (http://localhost:3000) are configured separately.

12.4 Refactoring data model for HTTP transport lab

Make the following set of changes:

`stock.v1.ts`

`stock-list.component.v1.ts`

`stock-list.component.v1.html`

`stock-item.component.v1.ts`

`stock-item.component.v1.html`

`create-stock.component.v1.ts`

12.5 Refactoring data model for HTTP transport explanation

12.6 Creating a proxy file to support CORS to access server on the same machine

Issue of CORS:
https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

Currently, the Angular app is served from localhost:4200. If the app attempts to make a HTTP request to localhost:3000 (where the local REST service is running on)

Create a file `proxy.conf.json` in the project root folder and populate it with:

```
{
  "/api": {
    "target": "http://localhost:3000",
    "secure": false
  }
}
```

In the main project folder, type:

ng serve --proxy-config proxy.conf.json

12.7 Creating a proxy file for serving the app explanation

12.8 Adding HTTP headers lab

`stock.service.v2.ts`

12.9 Adding HTTP headers explanation

12.10 Using observe and responseType properties lab

`stock.service.v3.ts`

`stock-list.component.v2.ts`

12.11 Using observe and responseType properties explanation

12.12 HTTP: Exercise

Video link: https://youtu.be/HPH7ki9S8u4

Run npm install in `exercise-http-basic-start`.

This is the code base for the solution of `exercise-services`.

Also run npm install in `exercise-http-server` to install the modules for the backend Express.js server. Start the server with

node index.js

Check that you can retrieve the initial list of employees from it from a browser with this URL:

`http://localhost:3000/api/employee`


```
[{"name":"Peter","age":35,"role":"Developer","email":"peter@gmail.com","languages":"java   python
html","os":""},{"name":"Jane","age":22,"role":"Admin","email":"jane@gmail.com","languages":"","os
":"windows
linux"},{"name":"Steven","age":28,"role":"Developer","email":"steven@gmail.com","languages":"pyth
on
javascript","os":""},{"name":"Ann","age":38,"role":"Admin","email":"ann@gmail.com","languages":""
,"os":"macos windows"},
```


Implement the following functionality:

a) Refactor the existing `EmployeeService` so that it interacts with the server to obtain the list of employees. The server exposes 3 APIs

- GET on `/api/employee` to get the current list of employees
- POST on `/api/employee` with the complete details of a new employee as a body to add to the current list
- PATCH on `/api/employee/:name` with the employee name in the URL and the complete details for the employee (age, email, role, languages and os) to change this for the targeted employee

b) Clicking on Show employee list will retrieve the list of employees from the backend and display them.

c) Details of a new employee can be entered in the employee form section as normal. The record is then sent to the backend and a subsequent refresh of the employee list should show that it is there. Provide an alert as well to indicate the status of attempting to add the new employee (either success or rejected because of the existence of the new employee)

d) Clicking on a specific employee listing in the employee list result in the appearance of a modify selected employee button

e) Clicking on this button will result in a new form page similar to creating a new employee form, but without the option of modifying the name.

f) Clicking on modify record will send the modification details to the backend and an alert will be provided to the success or otherwise of this attempt

g) Subsequent refresh of the employee list should show the updated details for the targeted employee

Remember to start the app with a reference to the proxy:

ng serve --proxy-config proxy.conf.json

You can run multiple instances of the development server `ng serve` to serve up different Angular apps at the same time by running them on different ports. By default, just typing `ng serve` starts the server on the default port 4200. You could then run another instance, for e.g. in a different project or workspace folder, by typing:

ng serve --port=xxxx --proxy-config proxy.conf.json

where xxxx can be any free port on your machine, e.g. 9999

To see the list of used ports, type netstat -an into a command prompt shell.

Action steps for `exercise-http-basic-start`:

Check Network tab in Developer tools for HTTP message interaction with backend:

No.	Action
1.	Import the necessary classes for performing HTTP into EmployeeService. Inject HTTPClient as a service.
2.	Refactor `getEmployees()` and `addEmployee()` to make appropriate HTTP calls and return appropriate types
3.	Refactor EmployeeDisplayComponent to create a suitable type for `EmployeeList` to hold the data returned from `getEmployees`. Refactor the component's template as well so it can properly display the data returned from this call. Test this out
4.	Refactor `createEmployee`() in CreateEmployeeForm so that it adds an employee correctly, and provides an error message if the employee to be added already exists in the backend database.
5.	Add two boolean properties to EmployeeDisplayComponent • `showList`: • `showModifyButton` Include a `Modify Selected Employee` button in the template, and use `showModifyButton` to control the appearance of this button. This button should be initially hidden from view.
6.	Modify its template to keep track of each item in the employee array being rendered using *NgFor directive. Register an event binding so that when the user clicks on a specific

	employee item in the list, the item's position in the array is passed as a parameter to the method `onSelectEmployee()` in EmployeeDisplayComponent.
7.	Add a property `selectedEmployee` to EmployeeDisplayComponent. Implement `onSelectEmployee()` to set `selectedEmployee` to the employee at the position passed in as parameter from previous step. Use `showModifyButton` to make the `Modify Selected Employee` disappear.
8.	Create a new component EmployeeModifyComponent. Add in an `@Input` property `myEmployee` of type Employee and `@Output` property `modifiedSuccess` of type Boolean. Initialize the output property correctly and inject in EmployeeService correctly.
9.	Update the EmployeeDisplayComponent template so that it shows the EmployeeModifyComponent template at the appropriate time. Use the `showList` boolean property in EmployeeDisplayComponent along with the use of the NgIf directive to flip between the display of the compete list of employees and EmployeeDisplayComponent. This property can be modified within a click event binding registered on the `Modify Selected Employee` in the template.
10	Pass the selected employee selectedEmployee to EmployeeModifyComponent via component binding in the template of EmployeeDisplayComponent. Create a new template for EmployeeModifyComponent so that it is nearly identical to the EmployeeFormComponent template, except for the following: • the name of the employee is shown in the form heading itself and is not available for modification in the form itself. • all the input fields in the form should be bound using NgModel to the corresponding properties in the `@Input` property `myEmployee` • the button at the end of the form should read `Modify Record` • the `ngSubmit` event should call `modifyEmployee()` in EmployeeModifyComponent
11	Implement an `updateEmployee()` method in EmployeeService which accepts an Employee type parameter, and use the HTTP Patch method to update the corresponding Employee record at the backend with the information from this parameter.
12	Implement `modifyEmployee()` in EmployeeModifyComponent so it uses the method implemented in the previous step and then emits a true using the `@Output` property `modifiedSuccess` when done. Ensure you cater for error conditions.
13	Provide a child event binding in template of EmployeeDisplayComponent to the triggering of the boolean event from the previous step so that the Modify Employee button is no longer shown, and the list of employees appears again.

13 HTTP Interceptor

Start from the code base in `http-interceptor\http-interceptor-start`. This is the final code base from the `http-basic` lab and can also be found in `http-basic\http-basic-final`

In the folder `http-basic\basic-server`
run:

```
npm install
node index.js
```

In the main project folder, type:

```
ng serve --proxy-config proxy.conf.json
```

13.1 Generate and register authorization service lab

In `src/app`
```
ng generate service services/auth
```

`app.module.v1.ts`

`auth.service.v1.ts`

13.2 Generate and register authorization service explanation

13.3 Refactor code and service to use the Auth service lab

`stock.service.v1.ts`

`stock-list.component.v1.ts`

`stock-list.component.v1.html`

13.4 Refactor code and service to use the Auth service explanation

13.5 Create and set up interceptor in root module lab

Create:
`src/app/services/stock-app.interceptor.ts`

`stock-app.interceptor.v1.ts`

`app.module.v2.ts`

13.6 Create and set up interceptor in root module explanation

13.7 Implement interception and response handling in interceptor lab

`stock-app.interceptor.v2.ts`

13.8 Implement interception and response handling in interceptor explanation

14 Routing

Start from the code base in `routing\routing-start`

In the folder `routing\second-server`
run:

npm install
node index.js

In the main project folder, type:

ng serve --proxy-config proxy.conf.json

14.1 Creating a routing module and linking it to root module lab

`index.v1.html`

In `src/app`
ng generate module app-routes --flat --routing

`app-routes-routing.module.v1.ts`

`app.module.v1.ts`

14.2 Modify main template to load components for routes lab

app.component.v2.html

app.component.v1.css

14.3 Providing default routes in routing module lab

app-routes-routing.module.v3.ts

14.4 Using route parameters

app-routes-routing.module.v4.ts

stock-details.component.v1.ts

14.5 Route navigation within component

register.component.v1.ts

login.component.v1.ts

14.6 Route navigation in template

stock-item.component.v1.html

stock-details.component.v1.html

14.7 Passing query parameters between components

login.component.v2.ts

stock-list.component.v1.ts

14.8 Navigating to the same page

stock-list.component.v1.html

stock-list.component.v2.ts

14.9 Generating authentication guard and updating modules lab

In `src/app`
ng generate guard guards/auth

and press spacebar and enter to select the default: CanActivate

`auth.guard.v1.ts`

`app.module.v2.ts`

`app-routes-routing.module.v5.ts`

14.10 Generating deactivation guard and updating modules lab

In `src/app`
ng generate guard guards/CreateStockDeactivate

and press spacebar and enter to select the default: CanActivate

`create-stock-deactivate.guard.v1.ts`

`app.module.v3.ts`

`app-routes-routing.module.v6.ts`

14.11 Generating route resolver service and updating modules lab

In `src/app`
ng generate service resolver/stock-load-resolver

and update with:

`stock-load-resolver.service.v1.ts`

`app.module.v4.ts`

`app-routes-routing.module.v7.ts`

`stock-details.component.v2.ts`

14.12 Routing: Exercise

Video link: https://youtu.be/GgSeXlvBaSM

Run npm install in `exercise-routing-start`.

This is the code base for the solution of `exercise-http-basic`.

Also run npm install in `exercise-http-server` to install the modules for the backend Express.js server. Start the server with

node index.js

Remember to start the app with a reference to the proxy:

ng serve --proxy-config proxy.conf.json

Refactor the code base to use proper routes for navigating the app:

a) There should be ideally 3 main routes (List Employees, Highlight Employees and Create Employee) visible and accessible from normal links on the main landing page. There is a 4th route Update Employee which is accessible from the List Employees route.

b) All other manually typed URLs should redirect to the main landing page

c) When the user completes entering a valid record in the Create Employee route, it will automatically navigate to the List Employees route

d) In the Highlight Employees route, there should be a button to confirm the string to do the highlighting on, and then there will be automatic navigation to the List Employees route where the relevant records are highlighted in the usual way

e) Clicking on a particular record in the employee list in the List Employees route will result in an Update Selected Employee button appearing. Clicking on this button will navigate to the Update Employee route where the details of the selected employee (except for the name) are available for editing.

f) Upon confirmation of the modified record, there will be automatic navigation back to the List Employees route.

g) The Modify Employee route contains an activation to ensure that it can only be navigated to by clicking on the Update Selected Employee button in the List Employees route, rather than actually typing the URL manually into the address bar.

h) All routes incorporate some logical form of basic protection against deactivation.

- Create Employee: if the employee form is visible, then the user is prompted

- Highlight Employees: if something has been typed in the textbox and the confirmation button is not clicked, then the user is prompted
- Update Employee: if something is typed into any of the textboxes and the confirmation button is not clicked, then the user is prompted
- List Employees: if a particular user has been selected and the Update Selected Employee button is not clicked, then the user is prompted

Hints

Once you have create the routing-specific module (`app-routes-routing.module.ts`), Angular will no longer automatically add import statements into the main module (`app.module.ts`) every time you generate a new component as Angular is uncertain which module the imports should be placed in, since a component can only be declared and imported in one module (and subsequently exported again to other modules that need to use it). To explicitly specify for example that the import statement should be placed in the main module, type

ng generate component *xxxx* --module app

You can create a generic `DeactivateGuard` for use with any component:
https://www.concretepage.com/angular-2/angular-candeactivate-guard-example
This code is a bit more complex than creating component specific `DeactivateGuards`, but it can be easily reused which makes it more useful when there are many generic `DeactivateGuards` to be used with multiple components.

A criteria to decide whether to navigate away from the Modify Employee route is whether any of the fields have been changed (i.e. status – `NgDirty`). To do this, you could access the form controls from `ngForm` directly in the component

https://stackoverflow.com/questions/37093432/angular-2-template-driven-form-access-ngform-in-component

Alternatively, you could keep track of all the employee fields in the component once you receive it from the List Employees route, and then compare this with the respective form field values tracked by `NgModel` when an action is made to navigate away.

Remember to start the app with a reference to the proxy:

ng serve --proxy-config proxy.conf.json

You can run multiple instances of the development server `ng serve` to serve up different Angular apps at the same time by running them on different ports. By default, just typing `ng serve` starts the server on the default port 4200. You could then run another instance, for e.g. in a different project or workspace folder, by typing:

ng serve --port=xxxx --proxy-config proxy.conf.json

where xxxx can be any free port on your machine, e.g. 9999

To see the list of used ports, type netstat -an into a command prompt shell.

Action list to be created ????

15 Deploying

Once we have created a production build, you can upload the bundled assets and scripts generated from the build to a public server. Typically, this will be server within your company infra or an external cloud server instance. Many platforms now offer serverless deployment which greatly simplifies the process of deploying onto a public cloud by abstracting away the lower-level details of server management. An example is: https://zeit.co/ which we will use:

Install Zeit Now for deployment with:

```
npm install -g now
```

You can use any of the projects that you have developed earlier for this purpose. In the main project folder, type:

```
ng build --prod
```

Then switch to dist folder in the command prompt and deploy with now:

```
now
```

Enter a valid email address to complete the initial registration. Check the inbox for an email and click on the link to complete verification. A confirmation message is displayed at the prompt. Type

```
now
```

again to deploy the app and obtain the public URL to access it (this will be in the form https://xxxxx.now.sh)